



INSTITUTO DO EMPREGO
E FORMAÇÃO PROFISSIONAL

Técnico Especialista em Gestão de Rede e Sistemas Computacionais

Relatório de Projeto

BuyPy — Base de Dados para Loja Online de Livros e Eletrónicos

Mamadu Aliu Djaló

UC00603: Criar a estrutura de uma Base de Dados e Programar em SQL

João Galamba

03/08/2026

Sumário

1. Introdução	3
2. Objetivos	3
Stored Procedures	3
3. Análise de Requisitos	6
3.1 Requisitos Funcionais	6
3.2 Requisitos Não Funcionais	6
4. Modelo de Dados	6
4.1 Diagrama Entidade-Relacionamento (ERD)	6
5. Implementação	7
5.1 Criação de Tabelas	7
5.2 Triggers	7
5.3 Procedures	7
5.4 Views	8
6. Segurança e Utilizadores	8
7. Testes	8
8. Dificuldades e Soluções	9
9. Conclusão	10
10. Referências	11
Anexos	11

1. Introdução

O presente relatório documenta o desenvolvimento do projeto BuyPy, uma base de dados relacional destinada a suportar uma loja online especializada na venda de livros e produtos eletrónicos. O trabalho foi realizado no âmbito do módulo de **Criar a estrutura de uma Base de Dados e Programar em SQL**, com o objetivo de aplicar, num caso prático, os conceitos de modelagem relacional, integridade de dados, programação em SQL (triggers, procedures e views) e gestão de permissões de acesso.

O sistema de gestão de base de dados utilizado foi o MySQL 8, e o desenvolvimento foi realizado com recurso ao MySQL Workbench e ao Visual Studio Code, com controlo de versões através do Git.

2. Objetivos

Este projeto tem como objetivos específicos:

- Desenhar e implementar uma base de dados relacional em MySQL 8 para uma loja online de livros e eletrónicos
- Garantir a integridade dos dados através de chaves primárias e estrangeiras, constraints CHECK e triggers
- Automatizar regras de negócio, como a encriptação de passwords e o cálculo de IVA, através de triggers
- Disponibilizar procedures para as operações principais do negócio (consulta de catálogo, gestão de encomendas, relatórios)
- Definir perfis de acesso distintos (cliente, operador, administrador), com permissões adequadas a cada perfil.

Stored Procedures

Stored Procedure é um trecho de código Transact SQL armazenado no banco de dados, que pode receber parâmetros e retornar valores. Quando uma Stored Procedure é executada, é realizada uma compilação da mesma. Resumindo, Stored Procedure basicamente pode ser uma ou mais ações que são salvas no banco de dados e que podem ser solicitadas sempre que necessário.

As SP's podem ser utilizadas para muitos fins, como:

Executar comandos INSERT;

Executar comandos DELETE;

Executar comandos UPDATE;

Executar comandos SELECT;

Efetuar controle em lista de registros (Cursor);

Efetuar a execução de outras SP's.

Com o advento das stored procedures, que permitem ao SGBD lidar com conceitos que anteriormente só eram resolvidos mediante programação, há uma clara necessidade de uma maneira mais elegante de controlar erros e exceções. Há dois tipos de manipuladores suportados pelo MySQL: manipuladores EXIT, que encerram imediatamente o bloco BEGIN/END corrente; e manipuladores CONTINUE, que permitem ao processo continuar depois que as ações do manipulador foram executadas (o manipulador UNDO, que pode ser familiar a usuários de outros SGBDs ainda não é suportado).

É muito comum, em aplicações que utilizam **bancos de dados**, que ações sejam disparadas em resposta ou como consequência de outras, realizando operações de cálculo, validações e, em geral, surtindo alterações na base de dados.

Triggers

Triggers (“gatilhos” em português) são objetos do banco de dados que, relacionados a certa tabela, permitem a realização de processamentos em consequência de uma determinada ação como, por exemplo, a inserção de um registro.

Os **triggers** podem ser executados ANTES ou DEPOIS das operações de **INSERT**, **UPDATE** e **DELETE** de registros.

Triggers

Trigger é um bloco de comandos Transact-SQL que é executando quando ocorre um comando INSERT, DELETE ou UPDATE for executando em um banco de dados.

Alguns exemplos de aplicações:

Validações;

Rotinas;

Consistência de dados;

Análise de dados;

Alteração em tabelas que referenciam uma tabela que está sendo trabalhada.

Existem três aspectos importantes de um gatilho:

Evento: uma alteração que ativa a trigger;

Condição: consulta ou teste executado quanto a trigger é ativada; Ação: procedimento executando quando a condição é atendida.

Vamos agora as definições das palavras reservadas de uma Trigger: `CREATE TRIGGER CriarUsuario` é o comando para se criar uma nova Trigger. o Caso uma Trigger já esteja criada é possível alterar ela através do comando `ALTER TRIGGER Nome_Trigger` criada; o O comando para exclusão de uma Trigger é `DROP TRIGGER Nome_Trigger`; `ON Aluno AFTER INSERT, UPDATE` define em qual tabela a Trigger será criada e em quais condições ela será executada. Exemplo: o Em nossa Trigger, caso efetuarmos um `INSERT INTO ALUNO` ou `UPDATE ALUNO` essa Trigger será executada. `DECLARE @IdPessoa INT` e as demais declarações de variáveis são utilizadas para criar variáveis locais que serão utilizadas como apoio na estrutura da Trigger.

view

A **view** pode ser definida como uma tabela virtual composta por linhas e colunas de dados vindos de tabelas relacionadas em uma query (um agrupamento de `SELECT`'s, por exemplo). As linhas e colunas da view são geradas dinamicamente no momento em que é feita uma referência a ela.

Como já dito, a query que **determina uma view** pode vir de uma ou mais tabelas, ou até mesmo de outras views.

Podemos **realizar qualquer query por meio de views**, assim como alterar dados por meio delas, o que é feito com algumas restrições.

Vantagens das Views

Temos muitos motivos e vantagens para usarmos views em nossos projetos. A seguir são citados três que podem fazer a diferença:

- **Reuso:** as views são objetos de caráter permanente. Pensando pelo lado produtivo isso é excelente, já que elas podem ser lidas por vários usuários simultaneamente.
- **Segurança:** as views permitem que ocultemos determinadas colunas de uma tabela. Para isso, basta criarmos uma view com as colunas que achamos necessário que sejam exibidas e as disponibilizarmos para o usuário.

Simplificação do código: as views nos permitem criar um código de programação muito mais limpo, na medida em que podem conter um **SELECT** complexo. Assim, **criar views** para os programadores a fim de poupá-los do trabalho de criar SELECT's é uma forma de aumentar a produtividade da equipe de desenvolvimento.

3. Análise de Requisitos

3.1 Requisitos Funcionais

- Registo e autenticação de clientes e operadores
- Consulta do catálogo de produtos, distinguindo livros e produtos eletrónicos
- Criação e gestão de encomendas, com cálculo automático de subtotal, IVA e total
- Controlo de stock, com validação de quantidade disponível ao adicionar produtos a uma encomenda
- Registo de recomendações de produtos a clientes
- Consulta de relatórios de encomendas diárias e anuais

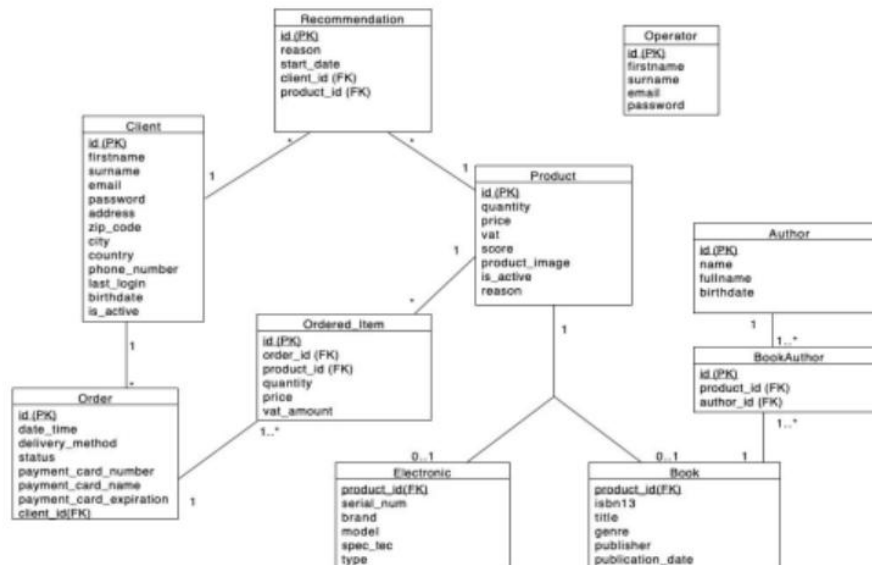
3.2 Requisitos Não Funcionais

- Segurança: as passwords de clientes e operadores nunca são guardadas em texto simples, sendo encriptadas com SHA2-256 através de triggers
- Integridade referencial: todas as relações entre tabelas são garantidas por chaves estrangeiras, com regras de CASCADE ou RESTRICT consoante o caso
- Validação de dados: formatos de email, número de telefone e ISBN-13 são validados antes da inserção
- Controlo de acesso: permissões diferenciadas consoante o perfil de utilizador (WEB_CLIENT, BUYDB_OPERATOR, BUYDB_ADMIN)

4. Modelo de Dados

4.1 Diagrama Entidade-Relacionamento (ERD)

O modelo é composto pelas entidades Client, Operator, Product, Book, Author, BookAuthor, Electronic, Recommendation, Orders e Ordered_Product. Product funciona como entidade-base, sendo especializada em Book e Electronic (relação de generalização/especialização 1:1). A relação entre livros e autores é modelada através da tabela associativa BookAuthor (muitos-para-muitos).



5. Implementação

5.1 Criação de Tabelas

Foram criadas 10 tabelas em InnoDB (para suportar integridade referencial): Client, Operator, Product, Book, Author, BookAuthor, Electronic, Recommendation, Orders e Ordered_Product. As relações entre Product e as suas especializações (Book, Electronic) usam ON DELETE CASCADE, garantindo que a remoção de um produto remove automaticamente os dados específicos associados. Foram aplicadas constraints CHECK para validar, por exemplo, que a quantidade em stock nunca é negativa, que o preço é sempre positivo e que o IVA se situa entre 0 e 100%.

5.2 Triggers

Foram implementadas as seguintes triggers para automatizar regras de negócio:

- `trg_client_before_insert` / `trg_client_before_update` — validam a robustez da password (mínimo 6 caracteres, maiúscula, minúscula, dígito e símbolo) e do número de telefone, encriptando a password com SHA2-256 antes de a guardar
- `trg_operator_before_insert` / `trg_operator_before_update` — aplicam a mesma lógica de validação e encriptação de password aos operadores
- `trg_product_before_insert` / `trg_product_before_update` — calculam automaticamente o valor de IVA (`vat_amount`) a partir do preço e da taxa de IVA
- `trg_book_before_insert` — valida o formato do ISBN-13 e confirma o dígito de controlo através do algoritmo oficial de checksum

5.3 Procedures

Foram implementadas as seguintes procedures, que suportam as principais operações do negócio:

Procedure	Função
ProductByType	Lista produtos filtrados por tipo (livro/eletrónico), incluindo preço, popularidade e nº de recomendações

DailyOrders	Lista as encomendas realizadas numa determinada data
AnnualOrders	Lista as encomendas de um cliente num determinado ano
CreateOrder	Cria uma nova encomenda associada a um cliente, devolvendo o ID gerado
GetOrderTotal	Calcula o subtotal, o valor de IVA e o total de uma encomenda
AddProductToOrder	Adiciona um produto a uma encomenda, valida o stock disponível e atualiza a quantidade em Product
AddBook	Regista um novo livro, criando o registo em Product, Book e, se necessário, um novo Author
AddElec	Regista um novo produto eletrónico, criando o registo em Product e Electronic

5.4 Views

View	Descrição
vw_product_catalog	Catálogo unificado de produtos (livros e eletrónicos) com dados comuns
vw_active_clients	Lista de clientes com estado ativo
vw_order_summary	Resumo de encomendas com subtotal, IVA e total calculados
vw_recommendations	Lista de recomendações com dados do cliente e do produto recomendado

6. Segurança e Utilizadores

Foram definidos três perfis de utilizador da base de dados, cada um com um nível de acesso adequado à sua função:

Utilizador	Permissões
WEB_CLIENT	Leitura geral do catálogo; pode registar-se e atualizar os próprios dados; pode criar e gerir as próprias encomendas através das procedures CreateOrder, GetOrderTotal e AddProductToOrder; apenas pode atualizar a quantidade de produtos
BUYDB_OPERATOR	Acesso de leitura e escrita a todas as tabelas e execução de todas as procedures, para gestão operacional do catálogo e encomendas
BUYDB_ADMIN	Acesso total à base de dados, incluindo gestão de permissões de outros utilizadores (WITH GRANT OPTION)

Todas as passwords de clientes e operadores são encriptadas com SHA2-256 antes de serem guardadas, nunca sendo armazenadas em texto simples.

7. Testes

A validação da base de dados foi realizada através da execução manual do script no MySQL Workbench e da inserção de dados de teste, confirmando o comportamento esperado das triggers, procedures e constraints.


```

.31 • SELECT 'Clientes' AS tabela, COUNT(*) AS total FROM `Client`
.32 UNION ALL SELECT 'Operadores', COUNT(*) FROM Operator
.33 UNION ALL SELECT 'Livros', COUNT(*) FROM Book
.34 UNION ALL SELECT 'Eletronicos', COUNT(*) FROM Electronic
.35 UNION ALL SELECT 'Autores', COUNT(*) FROM Author
.36 UNION ALL SELECT 'BookAuthor', COUNT(*) FROM BookAuthor
.37 UNION ALL SELECT 'Encomendas', COUNT(*) FROM Orders
.38 UNION ALL SELECT 'Ordered_Product', COUNT(*) FROM Ordered_Product
.39 UNION ALL SELECT 'Recomendacoes', COUNT(*) FROM Recommendation;

```

Resultado

	tabela	total
▶	Clientes	10
	Operadores	3
	Livros	10
	Eletronicos	10
	Autores	10
	BookAuthor	12
	Encomendas	20
	Ordered_Product	30
	Recomendacoes	20

8. Dificuldades e Soluções

Durante o desenvolvimento, é a correção dos erros no MySQL Workbench isso levou me noite e noite para resolver este erros que foram identificados e corrigidos através de análise das mensagens de erro.

Outra dificuldade consistiu em compreender algumas mensagens de erro do MySQL Workbench, A resolução destes problemas exigiu a análise das mensagens apresentadas pelo sistema, a verificação dos objetos já existentes e a reorganização da ordem de execução do script.

Apesar destas dificuldades, foi possível concluir o projeto com sucesso, criando a estrutura da base de dados, as tabelas, os relacionamentos, os triggers, as procedures, as views e inserindo corretamente todos os dados. Este projeto permitiu consolidar conhecimentos sobre modelagem de bases de dados, linguagem SQL e administração de bases de dados em MySQL, bem como desenvolver competências na resolução de problemas e na interpretação de mensagens de erro.

9. Conclusão

O projeto BuyPy permitiu aplicar, de forma integrada, os principais conceitos de modelagem e implementação de bases de dados relacionais: definição de entidades e relações, garantia de integridade através de chaves e constraints, automatização de regras de negócio com triggers, encapsulamento de lógica através de procedures e views, e definição de um modelo de permissões alinhado com diferentes perfis de utilizador.

Este projeto não está concluído, é um projeto para o futuro que tem muita coisa que pode ser melhorado e implementado na pratica com a criação da parte gráfica que permitirá o cliente utilizar para fazer as suas encomendas ou seja as suas comprar.

10. Referências

<https://www.devmedia.com.br/conceitos-e-criacao-de-views-no-sql-server/22390>

<https://www.devmedia.com.br/mysql-basico-triggers/37462>

<https://www.devmedia.com.br/advanced-series-mysql-stored-procedures/7301>

Anexos

Anexo A — Script SQL completo da base de dados BuyPy (ficheiro .sql em separado).